Implicit & Explicit Measures and Key DAX Concepts in Power BI

---

1. Introduction to DAX in Power BI

DAX (Data Analysis Expressions) is the formula language used in Power BI, Power Pivot, and Analysis Services Tabular models. It is designed to work with relational data and provides powerful capabilities to perform calculations, aggregations, filtering, time intelligence, and table manipulations. DAX consists of **functions**, **operators**, and **values** that allow you to build expressions for:

- **Measures**
- **Calculated columns**
- **Calculated tables**
- **Row-level security filters**

YATICORP
EMPOWERING EDUCATION

DAX is essential for creating advanced analytics by enabling:

- Mathematical operations
- Text manipulations
- Logical comparisons
- Statistical calculations
- Aggregations
- Time-based analysis
- Looping functions
- Table/row operations

Power BI offers over **1,000 DAX functions**, and Microsoft continues to update the list frequently. Official documentation is available at:
learn.microsoft.com/en-us/dax/, which also includes a downloadable PDF manual (over 1,200 pages).

This document focuses on **measures**, specifically **implicit and explicit measures**, and how they are used along with various DAX functions.

---

2. Measures in Power BI

A **measure** is a calculation used on data during report view visualization. Measures are evaluated at query time based on the filter context applied on visuals. Power BI provides two types of measures:

2.1 Implicit Measures

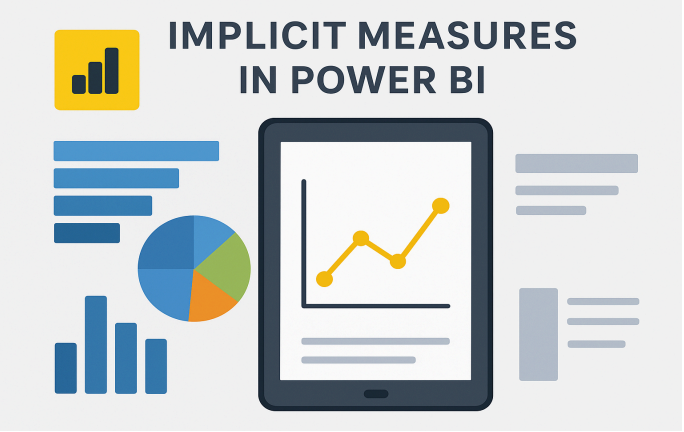
Implicit measures are created automatically by Power BI when you drag a numeric or categorical field into a visual. For example:

- Dragging **Quantity** into a Card visual → Power BI automatically performs **SUM(Quantity)**.
- Dragging **Order ID** into a Card → Power BI shows **First Order ID** by default.

YATICORP
EMPOWERING EDUCATION

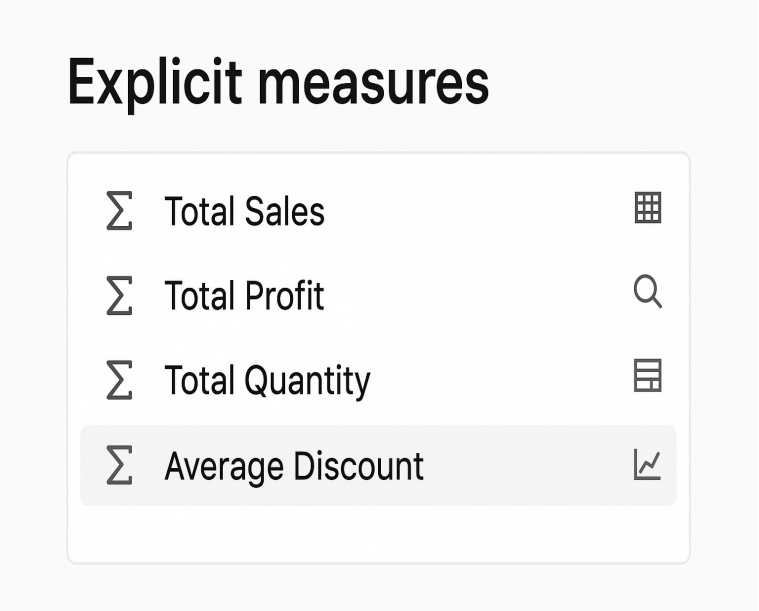
For implicit measures, Power BI automatically offers aggregation options such as:

- Sum
- Average
- Minimum
- Maximum
- Count
- Count Distinct
- Standard Deviation
- Variance
- Median



These implicit aggregations depend on the **data type** of the field.

2.2 Explicit Measures



Explicit measures are created manually by the user using DAX. They are reusable, customizable, and recommended for all professional report development.

To create an explicit measure:

1. Select the table where you want to store the measure.
2. Click **New measure** in the Modeling tab.
3. Enter the measure name and DAX expression.
4. Press Enter.

YATICORP
EMPOWERING EDUCATION

For example:

```
Total Quantity = SUM('Sales Data'[Quantity])
```

Explicit measures are essential for:

- Custom logic
- Complex calculations
- KPI development
- Financial ratios
- Time intelligence
- Consistent reporting
- Reusability across visuals

3. Comparing Implicit vs. Explicit Measures

Feature	Implicit Measure	Explicit Measure
Created by	Automatically by Power BI	Manually by user
Customization	Very limited	Fully customizable
Reusability	No	Yes
Formatting control	Limited	Full control
Best practice	Not recommended for enterprise	Always recommended
DAX expression	Not visible	Visible and editable

In professional BI solutions, **explicit measures are preferred** because they provide transparency, reusability, and version control.

---

4. Creating Basic Explicit Measures Using DAX

4.1 Total Quantity

Even if Power BI can automatically sum numerical fields, creating explicit versions ensures consistency.

YATICORP
EMPOWERING EDUCATION

```
Total Quantity = SUM('Sales Data'[Quantity])
```

This explicit measure performs the same operation as the implicit “Sum of Quantity” but can be reused anywhere and allows formatting options.

4.2 Count of Orders – Implicit Method

If you click on **Order ID** inside a Card visual:

- The default display will show **First Order ID**.
- From the dropdown (aggregation menu), you may select:
    - **Count**
    - **Count Distinct**

Since the same order ID may repeat multiple times (if multiple items are purchased in the same order), **Count Distinct** gives the number of unique orders.

4.3 Count of Orders – Explicit Measure

Using the **DISTINCTCOUNT()** DAX:

```
Number of Orders = DISTINCTCOUNT('Sales Data'[Order ID])
```

This explicit measure gives the same result as the implicit “Count Distinct” but is reusable.

YATICORP
EMPOWERING EDUCATION

4.4 Maximum Profit — Using Explicit Measure

```
Max Profit Amount = MAX('Sales Data'[Profit Amount])
```

This measure returns the highest profit value from the Profit Amount column.

4.5 Minimum Profit — Editing an Existing Measure

You can change the above measure to find the minimum value:

```
Min Profit Amount = MIN('Sales Data'[Profit Amount])
```

Power BI automatically updates the visuals after modifying the measure.

4.6 Implicit Maximum Profit

If you drag **Profit Amount** into a Card visual:

- It initially shows the Sum **of Profit Amount**.
- Using the dropdown → Change aggregation → **Max**.

This gives the same numerical result as the explicit MAX() measure.

---

5. Understanding MAX(), MAXA(), and MAXX()

When typing "MAX" in the DAX editor, Power BI shows:

5.1 MAX()

- Returns the largest numeric value or largest string.
- Ignores text and logical values.

Usage:

```
MAX('Sales Data'[Profit Amount])
```

YATICORP
EMPOWERING EDUCATION

5.2 MAXA()

- Similar to MAX() but considers **logical values** and text that represent numbers.
- TRUE = 1, FALSE = 0.
- Rarely used in standard BI reporting.

5.3 MAXX()

- An iterator function used for row-by-row calculations in a table or expression.
- Useful when you need to compute something per row and then get the maximum.

Example:

```
MAXX('Sales Data', 'Sales Data'[Quantity] * 'Sales Data'[Profit Amount])
```

---

6. Calculating Profitability Percentage Using DAX

Profitability =
Total Profit / Total Revenue × 100

There are two ways to create this using DAX.

YATICORP
EMPOWERING EDUCATION

6.1 Method 1: Using Mathematical Operators

```
Profitability % =
SUM('Sales Data'[Profit Amount])
   /
SUM('Sales Data'[Revenue])
```

After creating the measure:

- Format it as **Percentage** using the modeling tab.

If Total Profit = 12,000 and Total Revenue = 2,14,000
Profitability = 5.59%

This is exactly what is shown in your example.

6.2 Method 2: Using the DIVIDE() Function

DIVIDE() is a safer function because it avoids errors when denominator = 0.

```
Profitability % =
DIVIDE(
    SUM('Sales Data'[Profit Amount]),
    SUM('Sales Data'[Revenue])
)
```

Optional 3rd argument allows providing alternative output if denominator is zero:

```
Profitability % =
DIVIDE(
    SUM('Sales Data'[Profit Amount]),
    SUM('Sales Data'[Revenue]),
    0
)
```

This prevents divide-by-zero errors.

YATICORP
EMPOWERING EDUCATION

---

7. Important DAX Concepts Demonstrated Through These Examples

7.1 Filter Context

Measures are calculated based on filters applied through:

- Slicers
- Page filters
- Report filters
- Visual filters

Examples:

- Total Quantity changes based on month selected.
- Maximum Profit changes based on category selection.

Measures do not store values—they compute on the fly.

7.2 Row Context (Relevant for Iterators)

Functions like **SUMX, AVERAGEX, MAXX** evaluate expressions row-by-row. In contrast, simple functions like **SUM, MAX** work on entire columns at once.

The examples you discussed rely on column-level aggregation and not row-by-row calculations.

7.3 Data Types and Aggregations

Power BI treats fields differently:

- **Numeric fields** → default aggregation = **SUM**
- **Categorical fields** → default aggregation = **First**, but options include **Count**, **Count Distinct**.

Understanding these default behaviors helps avoid errors.

YATICORP
EMPOWERING EDUCATION

7.4 Reusability of Measures

Explicit measures can be used:

- In multiple visuals
- Across pages
- In other DAX expressions
- While creating KPIs
- For formatting rules

For example:

```
Profitability Category =
IF([Profitability %] > 0.05, "High", "Low")
```

This referencing is not possible with implicit measures.

---

8. Types of DAX Functions (High-Level Overview)

While only a few were used in your explanation, Power BI supports many categories:

1. Aggregation Functions

- SUM
- AVERAGE
- MIN / MAX
- COUNT / DISTINCTCOUNT

2. Logical Functions

- IF
- AND / OR
- SWITCH

YATICORP
EMPOWERING EDUCATION

3. Filter Functions

- FILTER
- ALL
- CALCULATE (the most powerful DAX function)

4. Date and Time Functions

- DATEADD
- SAMEPERIODLASTYEAR
- TOTALYTD

5. Mathematical Functions

- DIVIDE
- ROUND
- ABS

6. Statistical Functions

- MEDIAN
- STDEV
- VAR

7. Table Manipulation Functions

- SUMMARIZE
- VALUES
- CROSSJOIN

YATICORP
EMPOWERING EDUCATION

The functions used in your examples belong to the **aggregation** and **mathematical** categories.

---

9. Summary of DAX Measures Covered

Purpose	DAX Used	Type
Total Quantity	SUM()	Explicit
Total Revenue	SUM()	Explicit
Number of Orders	DISTINCTCOUNT()	Explicit
Maximum Profit	MAX()	Explicit
Minimum Profit	MIN()	Explicit
Profitability %	SUM(), DIVIDE()	Explicit
Implicit Aggregations	SUM, AVERAGE, MIN, MAX, COUNT	Implicit

---

10. Why DAX Is So Powerful

DAX allows calculations beyond simple Excel formulas:

- Works dynamically with filters
- Allows relationship-based calculations
- Enables advanced business KPIs
- Integrates with complex data models
- Handles row-level and column-level operations
- Supports time-based intelligence (YTD, QTD, MTD)

Even a simple calculation like Profitability % becomes more powerful due to:

- Filter context
- Relationships
- Reusability

YATICORP
EMPOWERING EDUCATION

11. Best Practices When Creating DAX Measures

1. **Use explicit measures, not implicit ones**, for professionalism and consistency.
2. **Store measures in a dedicated measures table** for organization.
3. Use meaningful names such as:
    - Total Profit
    - Total Revenue
    - Profitability %
4. Format measures appropriately:
    - Currency
    - Percentage
    - Units
5. Avoid hardcoding values—always reference columns.
6. Use DIVIDE() instead of simple division.
7. Minimize row-level operations unless necessary.
8. Comment complex DAX with // for readability.

YATICORP
EMPOWERING EDUCATION

---

12. Conclusion

The concepts covered—implicit and explicit measures, basic aggregation functions, MAX/MIN operations, counting techniques, and profitability calculation—form the foundation of DAX usage in Power BI. Once these basics are mastered, you can move toward more advanced topics such as:

- Time intelligence
- CALCULATE() and filter modification
- Iterator functions (SUMX, AVERAGEX)
- Relationship-aware calculations
- Dynamic segmentation
- KPIs and forecasting

DAX is extremely powerful, and even simple expressions can transform the depth of analysis in Power BI reports. With thousands of functions, DAX continues to evolve, and Power BI developers continuously use these capabilities to create dynamic, interactive, and insightful dashboards.

YATICORP
EMPOWERING EDUCATION